Epic 3 Penetration Test — sec Branch

Security Plan Iteration 2, Section 6 scope: outbound redirect (R20) + Iteration 1 regression checks

Prepared for Vimal · 2026-09-16 · Target: local instance of the sec branch (commit 7b0d5b5) · Epic 4 and Epic 5 excluded — neither is merged to main yet

1. Scope and method

Your plan's Section 6 table lists four Iteration 2 test areas plus a regression pass. Of those, only **Epic 3 (VPP recommendation)** is actually live on `main` / `sec` — Epic 4 (discharge alerts) and Epic 5 (voice) both exist only on unmerged feature branches, so they're out of scope for a pentest of what's actually deployed. This report covers:

- **Outbound redirect (Epic 3 / R20)** — attempted against the real `safeProviderUrl` source, rendered in an actual browser DOM
- **Regression (Iteration 1)** — security headers, HTTP methods, CORS, error handling, rate limiting, all run against a real running instance of the backend (not in-process test transport)

TLS was not tested — this environment has no certificate to test against locally. Recommend a separate check against the deployed `app.mypowerplant.tech` origin.

Method, honestly stated: no Docker or Postgres is available in this environment, so the backend ran against SQLite with the schema built directly from the current models and seeded with the real methodology/provider data (plus one attacker-controlled row added only to this local test database — see Finding 3). The frontend check imported the actual `safeProviderUrl` function from its real source file via esbuild and executed it in headless Chrome — not a reimplementation of the logic under test.

2. Findings

R20 — Outbound provider link allow-list PASS

Six payloads were run through the real `safeProviderUrl()` and the actual DOM-rendering logic it gates (mirroring `VppSuggestions.tsx`'s `TermsDialog` footer exactly). One real, allow-listed provider href; five attack payloads, including the exact malicious href seeded into the live backend for Finding 3 below.

Case	Input href	Rendered
Legit: AGL (real seed href)	`https://www.agl.com.au/...`	clickable link
Attack: compromised/off-list domain	`https://mypowerplant-trusted-portal.evil.example/steal-session`	blocked
Attack: lookalike domain	`https://agl.com.au.evil.example/phish`	blocked
Attack: suffix trick, no dot boundary	`https://not-agl.com.au/`	blocked
Attack: javascript: URI	`javascript:fetch(...+document.cookie)`	blocked
Attack: allow-listed host over plain HTTP	`http://www.agl.com.au/...`	blocked

Screenshot of the rendered result (green = legitimate, allowed through; red = attack payloads, correctly blocked):



DOM-dump evidence (machine-checked, not just visual) confirmed the same result — see Appendix A.

Regression: security headers PASS

Live `curl` against a real `uvicorn` process (not in-process test transport):

```
GET /api/health
< HTTP/1.1 200 OK
< x-content-type-options: nosniff
< x-frame-options: DENY
< referrer-policy: strict-origin-when-cross-origin
< content-security-policy: default-src 'none'; frame-ancestors 'none'; base-uri 'none';
form-action 'none'
```

CSP contains no `unsafe-inline` or `unsafe-eval`. Headers also confirmed present on a 404 error path. HSTS confirmed conditional and correct: absent over plain HTTP, present (`max-age=31536000; includeSubDomains`) when `X-Forwarded-Proto: https` is sent — matching the CloudFront-fronted production setup.

Regression: CORS, HTTP methods, rate limiting, error disclosure PASS

- Hostile origin (`https://evil.example.com`) never reflected in `Access-Control-Allow-Origin` ; `Access-Control-Allow-Credentials` never sent at all.
- POST/PUT/PATCH/DELETE on `/api/health` and `/api/methodology` → `405` uniformly. `POST /api/profile` → `404` (route doesn't exist).
- Rate limiter engaged under real network load: 70 rapid requests produced a mix of `200` s and `429` s, with `Retry-After: 60` on the throttled responses.
- Oversized query string (3000 chars), a script tag in a query param, a path-traversal attempt, and a SQL-injection-shaped path all returned normal, safe responses (`200` / `404`) — no crash.
- A malformed JSON body (`{not json`) sent to `GET /api/methodology` was simply ignored — the route declares no body parameter, so nothing attempts to parse it. Response was a normal 200 with no internal detail leaked.

Finding: Alembic migration graph had two competing heads LOW — now fixed

Found while setting up this test environment, not by design — a genuine defect, not a simulated attack. Running `alembic upgrade head` against a fresh database failed outright:

```
ERROR: Multiple head revisions are present for given argument 'head'
```

Cause: this branch's own `003_drop_submissions.py` and main's `003_create_methodology_tables.py` both declared the same parent revision, creating a branch in the migration graph that git's file-level merge never flagged as a conflict (they're two different files). `003_drop_submissions` was fully redundant — its only effect (dropping the `submissions` table) is already done by main's `005_drop_user_data_tables`.

Impact (Availability): a fresh developer environment or a deployment re-running migrations from this branch would fail to come up at all. The existing test suite never caught this because it builds its schema directly from the SQLModel classes and never actually runs Alembic.

Fixed: removed `003_drop_submissions.py` in commit `7b0d5b5`; `alembic heads` now shows a single head. Already pushed.

3. CIA impact summary

Area	Confidentiality	Integrity	Availability
R20 outbound redirect	Protected — no path for a crafted href to become a clickable phishing/credential-theft link	N/A	N/A
Security headers / CORS	Protected — CSP/frame/referrer controls intact; no cross-origin credential leakage	N/A	N/A
Rate limiting	N/A	N/A	Protected against basic denial-of-wallet abuse (defense-in-depth; API Gateway is the authoritative control per `rate_limit.py`'s own docstring)
Error handling	Protected — no stack trace, module path, or DB detail observed in any response	N/A	N/A
Alembic multi-head	N/A	N/A	Was broken (fresh deploys/dev setups couldn't migrate); now fixed

4. Prioritised recommendations

1. **None required for R20 or the Iteration 1 regression set** — all checks passed against live, real-attack-payload testing.
2. **Done:** the Alembic multi-head issue is fixed and pushed (commit `7b0d5b5`). Worth a quick sanity check next time anyone merges a long-lived branch with its own migrations: run `alembic heads`, not just the test suite, since the test suite bypasses Alembic entirely.

3. **Follow-up, not urgent:** check TLS configuration against the deployed `app.mypowerplant.tech` origin directly — out of scope for this local run.
4. **Follow-up, not urgent:** re-run this same Epic 3 scope once Epic 4/5 land, and extend to R22 (calendar/.ics injection — already verified safe on the unmerged branch, worth reconfirming after merge) and R17-R19 (voice) once your teammate's Web Speech API decision is settled.

Appendix A — DOM-dump evidence (R20)


```
{
  "Legit: AGL (real, allow-listed seed href)": {
    "rendered": "link",
    "href": "https://www.agl.com.au/solar-batteries/batteries/virtual-power-plant"
  },
  "Attack: compromised/off-list domain (evil-vpp)": { "rendered": "text", "href": null },
  "Attack: lookalike domain (agl.com.au.evil.example)": { "rendered": "text", "href": null },
  "Attack: suffix trick (not-agl.com.au)": { "rendered": "text", "href": null },
  "Attack: javascript: URI": { "rendered": "text", "href": null },
  "Attack: allow-listed host over plain HTTP": { "rendered": "text", "href": null }
}
```
